

Project Report

Rock-Paper-Scissors AI-Powered Game using Machine Learning

Submitted in partial fulfillment of the requirements for
VITyarthi Project — B.Tech First Year (2025–26)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Submitted by:

Name: RAAG SHRI

Roll No: 25BAI10431

COURSE: Fundamentals of AI & ML

COURSE CODE: CSA2001

Under the Guidance of:

Mr. Satyendra Singh

(SCAI)

Abstract

This project is about building a Rock-Paper-Scissors game where the opponent is not just a random computer — it actually tries to learn how the player plays and then use that information to beat them. The AI inside the game uses two basic machine learning ideas: Markov Chain transition modeling, and frequency-based pattern analysis. Both methods work together so the AI can keep getting better the more rounds are played.

The full project is made in Python and has two ways to run it — one is a simple text-based terminal interface, and the other is a proper graphical window built using Tkinter. The code is divided into separate files for game logic, AI model, settings configuration, GUI, and statistics tracking, which makes the overall structure clean and easy to follow.

The main learning point from this project was understanding how a program can store past events and use them to predict future behavior — which is a concept that shows up a lot in real machine learning and data science work. Even though the game itself is simple, the approach behind it has real-world relevance.

Chapter 1: Introduction

1.1 What is This Project About?

Rock-Paper-Scissors is something everyone knows how to play. Two people choose one of three moves at the same time, and whoever picks the winning one gets a point. It sounds random, but humans are actually not very good at being random. We tend to repeat moves, fall into patterns, or react to what happened in the last round.

This project uses that idea — the fact that humans are predictable — and builds an AI that watches how the player plays over time. The more the player plays, the better the AI gets at guessing what move is coming next. This makes the game more challenging and interesting the longer you continue.

1.2 Motivation

During the first semester, we studied the basics of Python programming and got introduced to the concept of machine learning in one of our lectures. The idea of a program that could actually learn from data seemed really interesting, but also a bit hard to understand just from theory. So this project was a way to actually see that concept working in a small, simple example.

Instead of building something that uses big complex neural networks (which I honestly would not understand yet), the goal was to use simpler statistical ideas that are still genuinely considered machine learning techniques. Markov chains are used in real NLP and prediction systems, and frequency analysis is a basic form of pattern recognition.

1.3 Objectives

- Build a working Rock-Paper-Scissors game where a player can play against a computer
- Give the computer AI that actually learns from the player's moves over time
- Use Markov Chain modeling and frequency analysis as the core ML concepts

- Create both a terminal-based and a GUI version of the game
- Support multiple difficulty levels with different learning rates and randomness
- Track and display game statistics throughout the session

1.4 Scope of the Project

This is a mini-project made by a first-year student, so the scope is kept at a level that is genuinely learnable and implementable within a few weeks. The AI does not use any external ML libraries like TensorFlow or PyTorch — everything is written from scratch using Python's built-in data structures. The project focuses on getting a solid understanding of how prediction and pattern learning work at a fundamental level.

Chapter 2: Literature Review / Background

2.1 Classic Rock-Paper-Scissors

The game has been around for centuries and is originally from China and Japan. In terms of game theory, it is a zero-sum game where neither player has a dominant strategy if both play randomly. However, when humans play, they almost never play randomly — which is what makes the AI possible.

2.2 Markov Chains

A Markov Chain is a mathematical model that describes a system that moves from one state to another, where the next state depends only on the current state (not on any earlier history). In simple words: given what is happening right now, what is most likely to happen next?

For this game, each move a player makes (rock, paper, or scissors) is a 'state.' The AI watches which move tends to follow each previous move and builds a table of probabilities. For example, if the player picks rock 70% of the time after scissors, the AI learns that and predicts rock when the player just played scissors.

Markov Chains are widely used in speech recognition, predictive text on phones, financial modeling, and many other fields. For a first-year student, this was a great introduction to a genuinely important mathematical tool.

2.3 Frequency-Based Prediction

This is actually the simpler of the two methods. It just keeps a count of how many times the player chose each move across the whole game, and assumes the player will most likely choose their most-used move next. The AI then plays the move that would beat that prediction.

It is a form of statistical analysis and is related to concepts like mode in statistics and frequency tables. While simple, it works surprisingly well for players who have a strong favorite move.

2.4 Combining Both Methods

The project uses both methods and randomly picks between their predictions before making the final AI choice. This is a simple form of ensemble learning — using multiple models together to get a better overall result. The exact proportion of randomness also changes based on the difficulty setting chosen by the player.

Chapter 3: System Design

3.1 Overall Architecture

The project is divided into five separate Python files, each responsible for a specific part of the system. This kind of structure is called modular design and makes it easier to debug, test, or modify individual parts without breaking the rest of the project.

File Name	Role / Responsibility
main.py	Entry point — runs the terminal-based text game
game.py	Core game logic — scoring, move history, round results
ai_model.py	AI brain — Markov Chain + frequency prediction
gui.py	Graphical interface using Tkinter
config.py	Difficulty settings and configuration constants
stats.py	Statistics tracking module (planned expansion)

3.2 Game Flow

When the game starts, a new `RPSGame` object and a new `RPS_AI` object are created. The player then picks rock, paper, or scissors. Before responding, the AI first calls `get_ai_move()` which internally runs both the Markov prediction and the frequency prediction. It then picks one of them (randomly) and returns the move that would beat the predicted player move. The round result is calculated and scores are updated. The AI also calls `update_model()` to record what the player just did, so it can learn from it for future rounds. This loop continues until the player quits.

3.3 Difficulty Levels

The config.py file defines four difficulty levels, each with different parameters that control how 'smart' the AI acts:

Level	Pattern Length	Randomness	Learning Threshold
Easy	2 moves back	50% random	Starts after 8 games
Medium	3 moves back	20% random	Starts after 5 games
Hard	4 moves back	10% random	Starts after 3 games
Expert	5 moves back	5% random	Starts after 2 games

At Easy difficulty, the AI plays randomly half the time, so it is basically just a normal random opponent. At Expert difficulty, it starts learning after only 2 rounds and almost never plays randomly, which makes it very tough to beat consistently.

Chapter 4: Implementation

4.1 AI Model — ai_model.py

This is the most important file in the whole project. The RPS_AI class contains three main methods:

- `update_model(player_move)` — records the transition from the previous move to the current one in a nested dictionary
- `predict_next_move()` — looks at what move the player most often plays after their last move and returns that as the prediction
- `frequency_based(history)` — counts total move frequencies and returns the most common one
- `get_ai_move(player_history, game)` — combines both predictions and returns the counter move

The transitions dictionary stores data in the format

`transitions[last_move][current_move] = count`. For example,

`transitions['rock']['scissors'] = 5` would mean that after playing rock, the player played scissors five times.

4.2 Core Game Logic — game.py

The RPSGame class handles all the actual game rules. The win conditions dictionary makes it easy to check who wins any round:

```
win_conditions = {
    'rock': 'scissors',    # rock beats scissors
    'scissors': 'paper',   # scissors beats paper
    'paper': 'rock'        # paper beats rock
}
```

The `play_round()` method also keeps running histories of all player and AI moves, which the AI model uses for learning. The `get_stats()` method returns a summary dictionary with scores and total games played.

4.3 GUI — gui.py

The graphical interface is made using Python's built-in Tkinter library. No installation is needed for this. The window shows the current score at the top, the

result of the last round in the middle, and three colored buttons at the bottom for Rock, Paper, and Scissors. There is also an AI status label that shows what move the AI is currently predicting the player will make next — this is a fun way to see the learning happening in real time.

The window background uses a dark navy color (#0f172a) which gives it a modern feel, and each move button has a different color (red, blue, green) for easy identification.

4.4 Terminal Mode — main.py

The terminal version runs in a simple while loop. It shows a numbered menu, takes keyboard input, converts the number to a move name, plays the round, and prints the result. It also has a stats option that shows current scores and win percentage without exiting the game loop.

Chapter 5: Testing

5.1 How the Project Was Tested

Since this is a relatively small project, testing was mostly done manually by actually playing the game many times and checking if the AI was behaving the way it should. A few specific scenarios were checked:

5.2 Test Cases

Test Scenario	Expected Behavior	Result
Player always picks Rock	AI should start playing Paper after enough rounds	Pass
Player alternates Rock-Scissors	Markov model should detect pattern and predict correctly	Pass
First round with no history	AI should pick randomly (no data yet)	Pass
Reset button clicked	All scores and AI memory should clear completely	Pass
View stats with 0 games	Should show 'No games played' message, not crash	Pass
Invalid input in terminal mode	Should show error and ask again without crashing	Pass

5.3 Observations

When playing casually without thinking too hard about strategy, the AI started winning more consistently after about 10–15 rounds. When deliberately trying to be random (like using dice or random number generator to decide moves), the

AI's win rate dropped significantly. This confirmed that the AI is actually learning and not just getting lucky.

Chapter 6: Results and Discussion

6.1 What Worked Well

- The modular code structure made it easy to work on each part separately
- Combining two prediction methods (Markov + frequency) made the AI more robust than using just one
- The difficulty system gives the game real replay value — Easy feels fair, Expert feels genuinely hard
- The Tkinter GUI works well and requires no external installation
- Real-time AI prediction display in the GUI is an interesting visual feature

6.2 Limitations

- The current implementation only looks at individual moves, not sequences of length 2 or more (full pattern_length from config is not yet used in the Markov model)
- stats.py is created as a file but is currently empty — statistics functionality is only partially implemented
- There is a duplicate method definition in gui.py (play() is defined twice) which is a bug that needs fixing
- The main.py uses a method name get_ai_move_with_randomness() but the AI class only has get_ai_move() — this would cause a crash in terminal mode
- No data persistence — when the game is closed, all learned patterns are lost

6.3 What I Learned

This project taught me several things that go beyond just writing Python code. The biggest takeaway was understanding how a machine learning model is basically just a system that stores past data in a useful structure and then uses that data to make predictions. Before this project, ML seemed like something extremely complex and abstract. Now I can see that at its core, even a simple statistical model counts as ML.

I also learned about the importance of code organization. When everything is in one file, it is hard to find bugs. Splitting the project into separate files made

debugging much easier. I also practiced using Python dictionaries, the Counter class from collections, and Tkinter widgets.

Chapter 7: Conclusion and Future Work

7.1 Conclusion

This project successfully demonstrates how a basic machine learning approach can be applied to a simple game to create a genuinely adaptive opponent. The AI uses Markov Chain transition modeling and frequency-based prediction to identify patterns in the player's behavior and choose counter-moves accordingly. The project is fully functional as a GUI application and mostly functional as a terminal application (with a small method name bug to fix).

For a first-year mini-project, this covers meaningful technical ground — data structures, OOP in Python, basic probability, pattern recognition, and GUI development — and produces something that is actually fun to interact with.

7.2 Future Improvements

- Fix the bug in main.py (wrong method name call) to make terminal mode fully working
- Implement full pattern-length Markov chain (looking at sequences of 2-5 previous moves instead of just 1)
- Complete the stats.py module with charts using matplotlib showing move frequencies and win trends
- Add save/load functionality using a JSON file so AI learning persists between sessions
- Implement the sound effects mentioned in config.py using pygame or tkinter's built-in audio
- Add a history panel in the GUI showing the last 10 rounds
- Potentially try a neural network version using PyTorch once higher-level ML topics are covered

References

1. Norvig, P. & Russell, S. (2020). Artificial Intelligence: A Modern Approach. Pearson — for basic AI concepts and game theory.
2. Python Software Foundation. (2024). Python 3 Documentation — collections module, defaultdict, Counter. docs.python.org
3. Python Software Foundation. (2024). Tkinter Documentation — GUI programming in Python. docs.python.org/3/library/tkinter.html
4. Wikipedia.— for background on Markov chain concepts.
5. GeeksforGeeks. Rock Paper Scissors Game in Python. [geeksforgeeks.org](https://www.geeksforgeeks.org) — referenced for basic game structure ideas.